KO

new 연산자와 생성자 함수

객체 리터럴 `{...}` 을 사용하면 객체를 쉽게 만들 수 있습니다. 그런데 개발을 하다 보면 유사한 객체를 여러 개 만들어야 할 때가 생기곤 합니다. 복수의 사용자, 메뉴 내 다양한 아이템을 객체로 표현하려고 하는 경우가 그렇죠.

`'new'` 연산자와 생성자 함수를 사용하면 유사한 객체 여러 개를 쉽게 만들 수 있습니다.

생성자 함수

생성자 함수(constructor function)와 일반 함수에 기술적인 차이는 없습니다. 다만 생성자 함수는 아래 두 관례를 따릅니다.

1. 함수 이름의 첫 글자는 대문자로 시작합니다.
2. 반드시 `'new'` 연산자를 붙여 실행합니다.

예시:

```
function User(name) {
  this.name = name;
  this.isAdmin = false;
}

let user = new User("보라");

alert(user.name); // 보라
alert(user.isAdmin); // false
```

`new User(...)` 를 써서 함수를 실행하면 아래와 같은 알고리즘이 동작합니다.

1. 빈 객체를 만들어 `this` 에 할당합니다.
2. 함수 본문을 실행합니다. `this` 에 새로운 프로퍼티를 추가해 `this` 를 수정합니다.
3. `this` 를 반환합니다.

예시를 이용해 `new User(...)` 가 실행되면 무슨 일이 일어나는지 살펴 보도록 하겠습니다.

```
function User(name) {
  // this = {};  (빈 객체가 암시적으로 만들어짐)

  // 새로운 프로퍼티를 this에 추가함
  this.name = name;
```

```
  this.isAdmin = false;

  // return this;  (this가 암시적으로 반환됨)
}
```

이제 `let user = new User("보라")` 는 아래 코드를 입력한 것과 동일하게 동작합니다.

```
let user = {
  name: "보라",
  isAdmin: false
};
```

`new User("보라")` 이외에도 `new User("호진")`, `new User("지민")` 등을 이용하면 손쉽게 사용자 객체를 만들 수 있습니다. 객체 리터럴 문법으로 일일이 객체를 만드는 방법보다 훨씬 간단하고 읽기 쉽게 객체를 만들 수 있게 되었죠.

생성자의 의의는 바로 여기에 있습니다. 재사용할 수 있는 객체 생성 코드를 구현하는 것이죠.

잠깐! 모든 함수는 생성자 함수가 될 수 있다는 점을 잊지 마시기 바랍니다. `new` 를 붙여 실행한다면 어떤 함수라도 위에 언급된 알고리즘이 실행됩니다. 이름의 '첫 글자가 대문자'인 함수는 `new` 를 붙여 실행해야 한다는 점도 잊지 마세요. 공동의 약속이니까요.

new function() { ... }

재사용할 필요가 없는 복잡한 객체를 만들어야 한다고 해봅시다. 많은 양의 코드가 필요할 겁니다. 이럴 땐 아래와 같이 코드를 익명 생성자 함수로 감싸주는 방식을 사용할 수 있습니다.

```
let user = new function() {
  this.name = "John";
  this.isAdmin = false;

  // 사용자 객체를 만들기 위한 여러 코드.
  // 지역 변수, 복잡한 로직, 구문 등의
  // 다양한 코드가 여기에 들어갑니다.
};
```

생성자와 return문

생성자 함수엔 보통 `return` 문이 없습니다. 반환해야 할 것들은 모두 `this` 에 저장되고, `this` 는 자동으로 반환되기 때문에 반환문을 명시적으로 써 줄 필요가 없습니다.

그런데 만약 `return` 문이 있다면 어떤 일이 벌어질까요? 아래와 같은 간단한 규칙이 적용됩니다.

- 객체를 `return` 한다면 `this` 대신 객체가 반환됩니다.
- 원시형을 `return` 한다면 `return` 문이 무시됩니다.

`return` 뒤에 객체가 오면 생성자 함수는 해당 객체를 반환해주고, 이 외의 경우는 `this` 가 반환되죠.

아래 예시에선 첫 번째 규칙이 적용돼 `return` 은 `this` 를 무시하고 객체를 반환합니다.

```
function BigUser() {

  this.name = "원숭이";

  return { name: "고릴라" };  // <-- this가 아닌 새로운 객체를 반환함
}

alert( new BigUser().name );  // 고릴라
```

아무것도 `return` 하지 않는 예시를 살펴봅시다. 원시형을 반환하는 경우와 마찬가지로 두 번째 규칙이 적용됩니다.

```
function SmallUser() {

  this.name = "원숭이";

  return; // <-- this를 반환함
}

alert( new SmallUser().name );  // 원숭이
```

`return` 문이 있는 생성자 함수는 거의 없습니다. 여기선 튜토리얼의 완성도를 위해 특이 케이스를 소개해보았습니다.

괄호 생략하기

인수가 없는 생성자 함수는 괄호를 생략해 호출할 수 있습니다.

```
let user = new User; // <-- 괄호가 없음
// 아래 코드는 위 코드와 똑같이 동작합니다.
let user = new User();
```

명세서엔 괄호를 생략해도 된다고 정의되어 있지만, '좋은 스타일'은 아닙니다.

생성자 내 메서드

생성자 함수를 사용하면 매개변수를 이용해 객체 내부를 자유롭게 구성할 수 있습니다. 엄청난 유연성이 확보되죠.

지금까진 `this` 에 프로퍼티를 더해주는 예시만 살펴봤는데, 메서드를 더해주는 것도 가능합니다.

아래 예시에서 `new User(name)` 는 프로퍼티 `name` 과 메서드 `sayHi` 를 가진 객체를 만들어줍니다.

```
function User(name) {
  this.name = name;

  this.sayHi = function() {
    alert( "제 이름은 " + this.name + "입니다." );
  };
}

let bora = new User("이보라");

bora.sayHi(); // 제 이름은 이보라입니다.

/*
bora = {
   name: "이보라",
   sayHi: function() { ... }
}
*/
```

class 문법을 사용하면 생성자 함수를 사용하는 것과 마찬가지로 복잡한 객체를 만들 수 있습니다. class에 대해선 추후 학습하도록 하겠습니다.

요약

- 생성자 함수(짧게 줄여서 생성자)는 일반 함수입니다. 다만, 일반 함수와 구분하기 위해 함수 이름 첫 글자를 대문자로 씁니다.
- 생성자 함수는 반드시 `new` 연산자와 함께 호출해야 합니다. `new` 와 함께 호출하면 내부에서 `this` 가 암시적으로 만들어지고, 마지막엔 `this` 가 반환됩니다.

생성자 함수는 유사한 객체를 여러 개 만들 때 유용합니다.

자바스크립트는 언어 차원에서 다양한 생성자 함수를 제공합니다. 날짜를 나타내는 데 쓰이는 `Date` , 집합(set)을 나타내는 데 쓰이는 `Set` 등의 내장 객체는 이런 생성자 함수를 이용해 만들 수 있습니다. 자세한 내용은 다시 살펴보도록 하겠습니다.

과제

함수 두 개로 동일한 객체 만들기

중요도: 2

`new A()==new B()` 가 성립 가능한 함수 `A` 와 `B` 를 만드는 게 가능할까요?

```
function A() { ... }
function B() { ... }

let a = new A;
let b = new B;

alert( a == b ); // true
```

만약 가능하다면, 실행 가능한 예시를 작성해 보세요.

해답

계산기 만들기

중요도: 5

아래와 같은 세 개의 메서드를 가진 생성자 함수, `Calculator` 를 만들어보세요.

- `read()` – `prompt` 함수를 이용해 사용자로부터 값 두 개를 받고, 이를 객체 프로퍼티에 저장합니다.
- `sum()` – 프로퍼티에 저장된 값 두 개를 더한 후 반환합니다.
- `mul()` – 프로퍼티에 저장된 값 두 개를 곱한 후 반환합니다.

예시:

```
let calculator = new Calculator();
calculator.read();

alert( "Sum=" + calculator.sum() );
alert( "Mul=" + calculator.mul() );
```

데모 실행하기

테스트 코드가 담긴 샌드박스를 열어 정답을 작성해보세요.

해답

누산기 만들기

중요도: 5

생성자 함수 `Accumulator(startingValue)` 를 만들어 보세요.

`Accumulator(startingValue)` 를 이용해 만드는 객체는 아래와 같은 요건을 충족해야 합니다.

- 프로퍼티 `value` 에 현재 값(current value)을 저장합니다. 최초 호출 시엔 생성자 함수의 인수, `startingValue` 에서 시작값(starting value)을 받아옵니다.

- 메서드 `read()` 에선 `prompt` 함수를 사용해 사용자로부터 숫자를 받아오고, 받은 숫자를 `value` 에 더해줍니다.

프로퍼티 `value` 엔 `startingValue` 와 사용자가 입력한 모든 값의 총합이 더해져 저장됩니다.

데모를 위한 코드는 다음과 같습니다.

```
let accumulator = new Accumulator(1); // 최초값: 1

accumulator.read(); // 사용자가 입력한 값을 더해줌
accumulator.read(); // 사용자가 입력한 값을 더해줌

alert(accumulator.value); // 최초값과 사용자가 입력한 모든 값을 더해 출력함
```

데모 실행하기

테스트 코드가 담긴 샌드박스를 열어 정답을 작성해보세요.

해답